

CMPS455– Computer Security

Project Title : Automated Cryptanalysis System for Classical Ciphers

Name : Hadi Hussein Muselmani

Id: 202304319

Instructors name : Dr Mohamad al saleh

TA : Lynne Al arwadi

Introduction

Classical cryptography uses techniques such as Caesar, substitution, and transposition ciphers to protect information by converting plaintext into unreadable ciphertext. This project, titled **“Automated Cryptanalysis System for Classical Ciphers,”** aims to develop a smart system that can automatically detect the type of cipher used and recover the original plaintext without knowing the encryption key. The system applies cryptanalysis techniques such as frequency analysis, brute-force search, pattern recognition, and dictionary scoring to break encrypted messages. The project helps demonstrate the weaknesses of classical ciphers while providing practical knowledge in cryptography, cybersecurity, and automated attack methods.

Objective

The main objective of this project is to design and implement an automated cryptanalysis system capable of identifying and breaking classical ciphers such as Caesar cipher, monoalphabetic substitution cipher, and transposition cipher. The system analyzes encrypted text, predicts the cipher type, and applies suitable cryptanalysis techniques like frequency analysis, brute-force attacks, and pattern matching to recover the original plaintext automatically. The project also aims to demonstrate the weaknesses of classical encryption methods and provide practical understanding of cryptography and cybersecurity concepts.

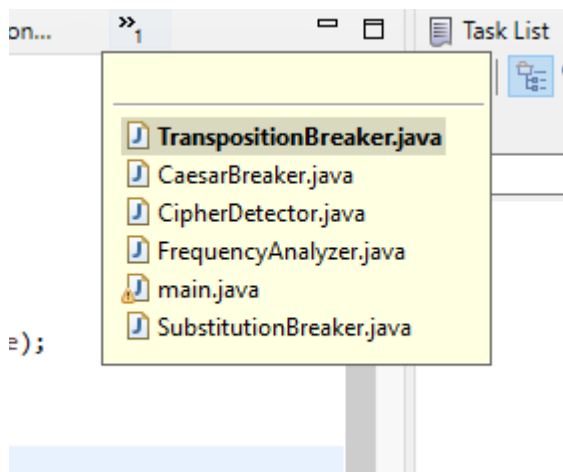
Methodology

This project was developed using Java in the Eclipse IDE. The system takes ciphertext as input, preprocesses it, and analyzes its patterns using frequency analysis and statistical techniques to detect the cipher type. Based on the detection result, the program applies suitable cryptanalysis methods such as brute-force attacks, frequency analysis, and key searching to recover the original plaintext. Finally, the system displays the detected cipher type, possible key, and decrypted message.

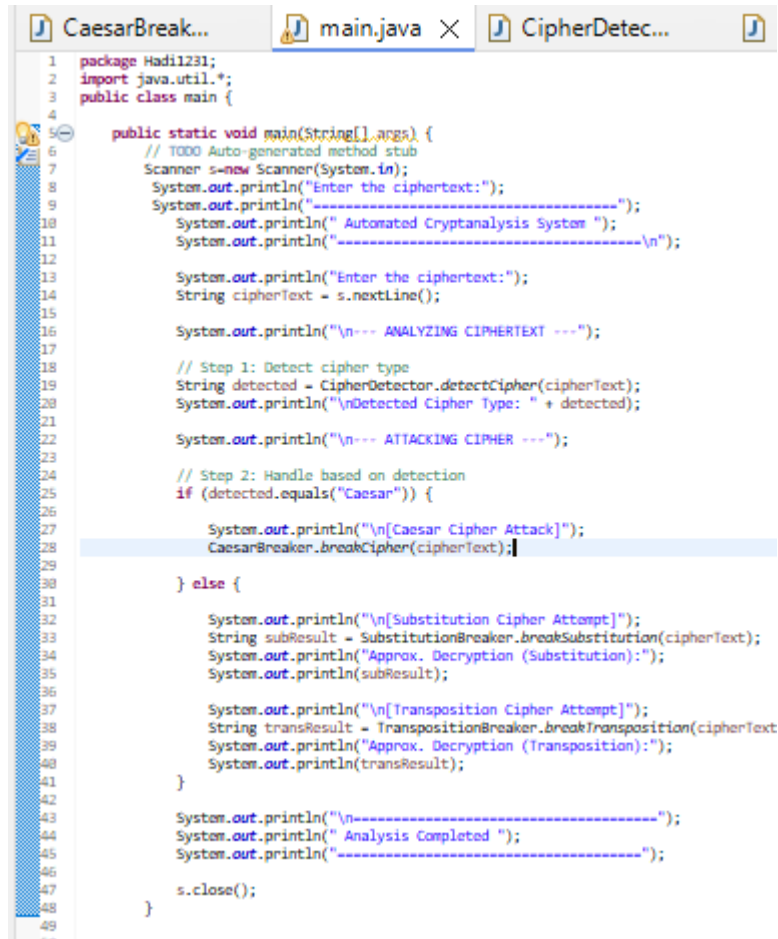
Implementation

The project was implemented using Java programming language in the Eclipse IDE. The system was developed using object-oriented programming concepts and different modules for cipher detection and cryptanalysis.

This project Consisted of 6 classes including a Main class for running the program.



1. Main class :



```
1 package Hadi1231;
2 import java.util.*;
3 public class main {
4
5     public static void main(String[] args) {
6         // TODO Auto-generated method stub
7         Scanner s=new Scanner(System.in);
8         System.out.println("Enter the ciphertext:");
9         System.out.println("-----");
10        System.out.println(" Automated Cryptanalysis System ");
11        System.out.println("-----\n");
12
13        System.out.println("Enter the ciphertext:");
14        String cipherText = s.nextLine();
15
16        System.out.println("\n--- ANALYZING CIPHERTEXT ---");
17
18        // Step 1: Detect cipher type
19        String detected = CipherDetector.detectCipher(cipherText);
20        System.out.println("\nDetected Cipher Type: " + detected);
21
22        System.out.println("\n--- ATTACKING CIPHER ---");
23
24        // Step 2: Handle based on detection
25        if (detected.equals("Caesar")) {
26
27            System.out.println("\n[Caesar Cipher Attack]");
28            CaesarBreaker.breakCipher(cipherText);
29
30        } else {
31
32            System.out.println("\n[Substitution Cipher Attempt]");
33            String subResult = SubstitutionBreaker.breakSubstitution(cipherText);
34            System.out.println("Approx. Decryption (Substitution):");
35            System.out.println(subResult);
36
37            System.out.println("\n[Transposition Cipher Attempt]");
38            String transResult = TranspositionBreaker.breakTransposition(cipherText);
39            System.out.println("Approx. Decryption (Transposition):");
40            System.out.println(transResult);
41        }
42
43        System.out.println("\n-----");
44        System.out.println(" Analysis Completed ");
45        System.out.println("-----");
46
47        s.close();
48    }
49 }
```

2. CaesarBreaker.java: performs brute-force attacks on Caesar cipher.

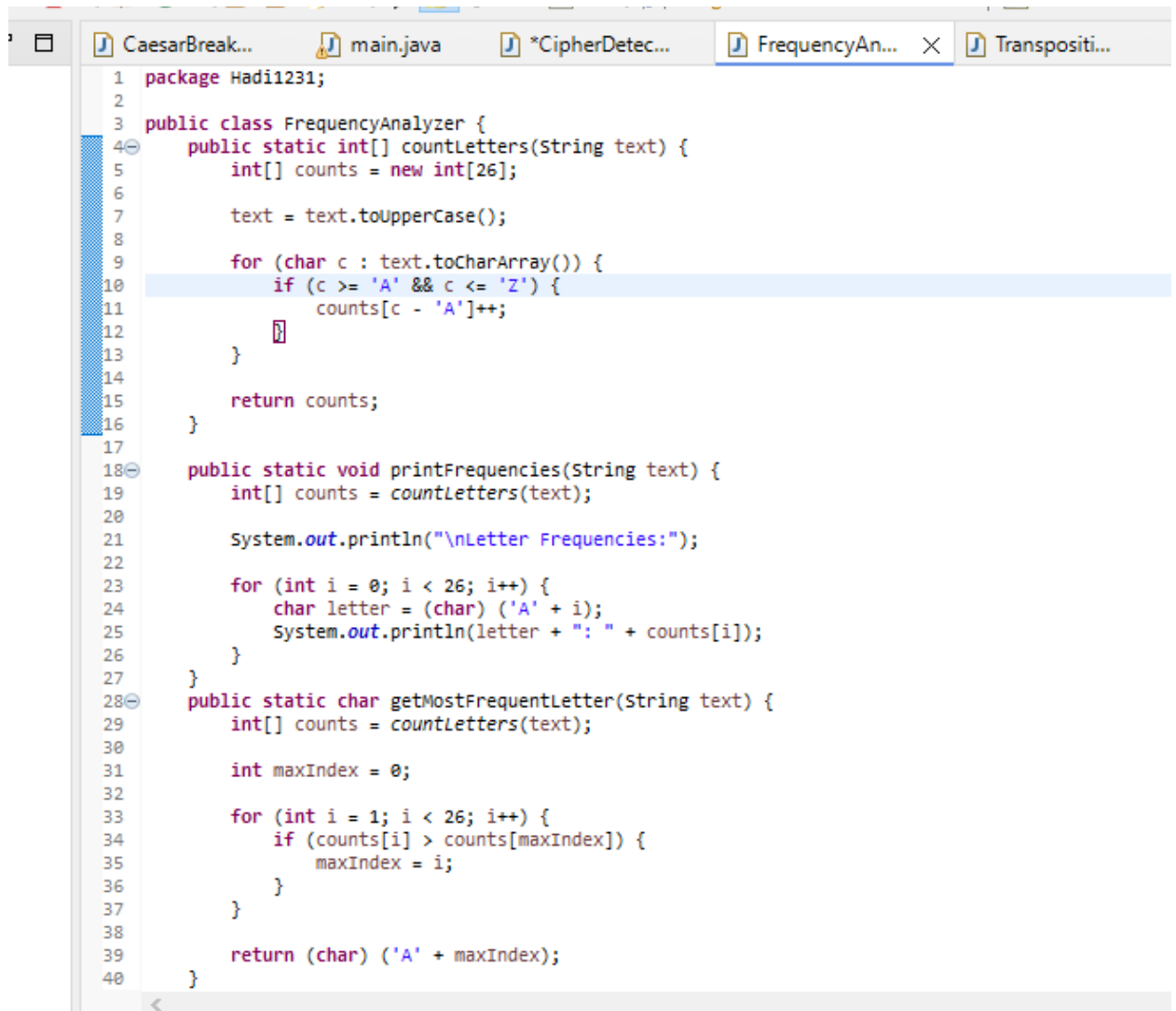
```
CaesarBreak... X main.java CipherDetec... FrequencyAn... Substituti
1 package Hadi1231;
2
3 public class CaesarBreaker {
4     public static String decrypt(String text, int shift) {
5         StringBuilder result = new StringBuilder();
6
7         for (char c : text.toCharArray()) {
8             if (Character.isLetter(c)) {
9                 char base = Character.isUpperCase(c) ? 'A' : 'a';
10                char decrypted = (char) ((c - base - shift + 26) % 26 + base);
11                result.append(decrypted);
12            } else {
13                result.append(c);
14            }
15        }
16        return result.toString();
17    }
18    public static int scoreText(String text) {
19        String[] commonWords = {"THE", "AND", "IS", "IN", "TO", "OF"};
20        int score = 0;
21
22        String upperText = text.toUpperCase();
23
24        for (String word : commonWords) {
25            if (upperText.contains(word)) {
26                score += 10;
27            }
28        }
29
30        return score;
31    }
32    public static void breakCipher(String cipherText) {
33        int bestScore = -1;
34        String bestDecryption = "";
35        int bestKey = 0;
36
37        for (int shift = 1; shift < 26; shift++) {
38            String decrypted = decrypt(cipherText, shift);
39            int score = scoreText(decrypted);
40        }
41    }
}
```

3. CipherDetector.java : predicts the cipher type



```
1 package Hadi1231;
2
3 public class CipherDetector {
4     public static String detectCipher(String cipherText) {
5         int bestScore = -1;
6
7         for (int shift = 1; shift < 26; shift++) {
8             String decrypted = CaesarBreaker.decrypt(cipherText, shift);
9             int score = CaesarBreaker.scoreText(decrypted);
10
11             if (score > bestScore) {
12                 bestScore = score;
13             }
14         }
15
16         if (bestScore >= 10) {
17             return "Caesar";
18         } else {
19             return "Unknown";
20         }
21     }
22 }
23
24
```

4. FrequencyAnalyzer.java : This module analyzes the frequency of letters in the ciphertext



```
1 package Hadi1231;
2
3 public class FrequencyAnalyzer {
4     public static int[] countLetters(String text) {
5         int[] counts = new int[26];
6
7         text = text.toUpperCase();
8
9         for (char c : text.toCharArray()) {
10             if (c >= 'A' && c <= 'Z') {
11                 counts[c - 'A']++;
12             }
13         }
14
15         return counts;
16     }
17
18     public static void printFrequencies(String text) {
19         int[] counts = countLetters(text);
20
21         System.out.println("\nLetter Frequencies:");
22
23         for (int i = 0; i < 26; i++) {
24             char letter = (char) ('A' + i);
25             System.out.println(letter + ": " + counts[i]);
26         }
27     }
28
29     public static char getMostFrequentLetter(String text) {
30         int[] counts = countLetters(text);
31
32         int maxIndex = 0;
33
34         for (int i = 1; i < 26; i++) {
35             if (counts[i] > counts[maxIndex]) {
36                 maxIndex = i;
37             }
38         }
39
40         return (char) ('A' + maxIndex);
41     }
42 }
```

5. SubstitutionBreaker.java : This module is responsible for attacking monoalphabetic substitution ciphers

```
CaesarBreak... *CipherDetec... FrequencyAn... *Substituti... X Transpositi...
1 package Hadi1231;
2 import java.util.Arrays;
3 import java.util.Comparator;
4 public class SubstitutionBreaker {
5     private static final String ENGLISH_FREQ = "ETAOINSHRDLCEUMWFGYPBVKJXQZ";
6
7     public static String breakSubstitution(String cipherText) {
8         cipherText = cipherText.toUpperCase();
9
10        int[] counts = FrequencyAnalyzer.countLetters(cipherText);
11
12        Character[] letters = new Character[26];
13        for (int i = 0; i < 26; i++) {
14            letters[i] = (char) ('A' + i);
15        }
16
17        Arrays.sort(letters, new Comparator<Character>() {
18            public int compare(Character a, Character b) {
19                return counts[b - 'A'] - counts[a - 'A'];
20            }
21        });
22
23        char[] mapping = new char[26];
24        for (int i = 0; i < 26; i++) {
25            mapping[letters[i] - 'A'] = ENGLISH_FREQ.charAt(i);
26        }
27
28        StringBuilder result = new StringBuilder();
29
30        for (char c : cipherText.toCharArray()) {
31            if (c >= 'A' && c <= 'Z') {
32                result.append(mapping[c - 'A']);
33            } else {
34                result.append(c);
35            }
36        }
37    }
38}
```

6. TranspositionBreaker.java: This module attempts to break transposition ciphers by testing different key lengths and rearranging character positions until readable plaintext is produced.

```

1 package Hadi1231;
2
3 public class TranspositionBreaker {
4     public static String breakTransposition(String cipherText) {
5         String bestResult = cipherText;
6         int bestScore = -1;
7
8         for (int key = 2; key <= 6; key++) {
9             String attempt = decrypt(cipherText, key);
10            int score = CaesarBreaker.scoreText(attempt);
11
12            System.out.println("Key " + key + ": " + attempt + " | Score: " + score);
13
14            if (score > bestScore) {
15                bestScore = score;
16                bestResult = attempt;
17            }
18        }
19
20        return bestResult;
21    }
22
23    public static String decrypt(String text, int key) {
24        int numRows = (int) Math.ceil((double) text.length() / key);
25        char[][] grid = new char[numRows][key];
26
27        int index = 0;
28
29        for (int col = 0; col < key; col++) {
30            for (int row = 0; row < numRows; row++) {
31                if (index < text.length()) {
32                    grid[row][col] = text.charAt(index++);
33                }
34            }
35        }
36
37        StringBuilder result = new StringBuilder();
38        for (int row = 0; row < numRows; row++) {
39            for (int col = 0; col < key; col++) {
40
41                result.append(grid[row][col]);
42            }
43        }
44
45        return result.toString();
46    }
47 }

```

Results and Output

Test 1 :

```
Problems @ Javadoc Declaration Console X
<terminated> main [Java Application] C:\Users\HP\.p2\pool\plu
Enter the ciphertext:
XLMW MW E XIWX

--- ANALYZING CIPHERTEXT ---

Detected Cipher Type: Caesar

--- ATTACKING CIPHER ---

[Caesar Cipher Attack]
Detected Cipher: Caesar
Best Key: 4
Decrypted Text: THIS IS A TEST
Confidence Score: 10

=====
Analysis Completed
=====
```

Test 2 :

```
Problems @ Javadoc Declaration Console X
<terminated> main [Java Application] C:\Users\HP\.p2\pool\plugin
Enter the ciphertext:
=====
Automated Cryptanalysis System
=====

Enter the ciphertext:
HLOOLELWRD
|
--- ANALYZING CIPHERTEXT ---

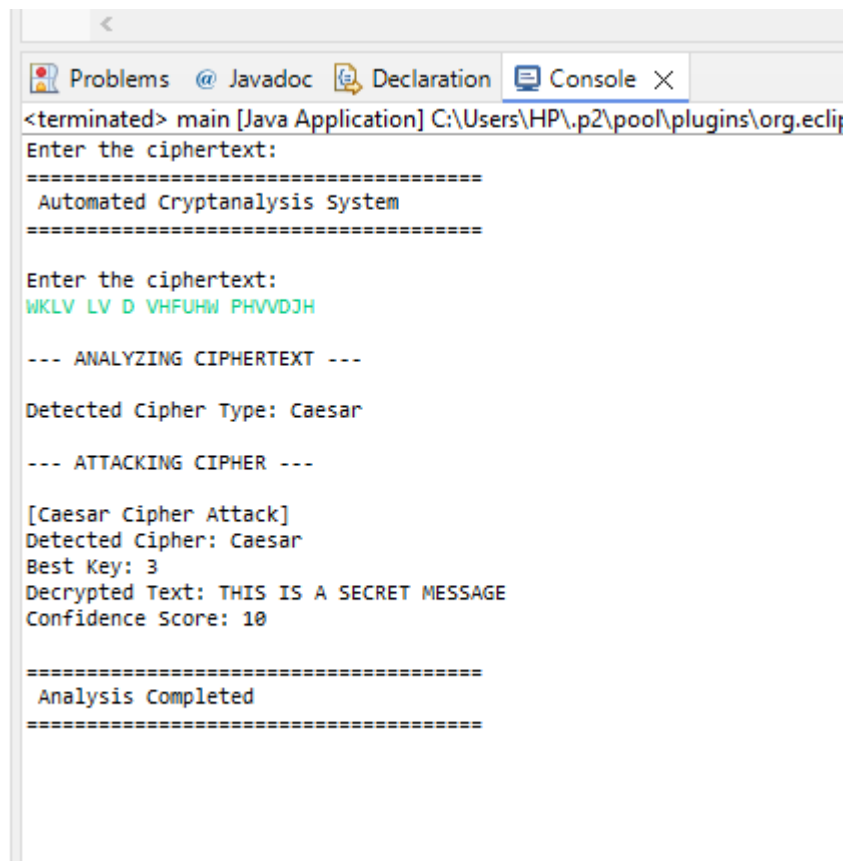
Detected Cipher Type: Caesar

--- ATTACKING CIPHER ---

[Caesar Cipher Attack]
Detected Cipher: Caesar
Best Key: 3
Decrypted Text: EILLIBITOA
Confidence Score: 10

=====
Analysis Completed
=====
```

Test 3 :



```
<terminated> main [Java Application] C:\Users\HP\p2\pool\plugins\org.eclij
Enter the ciphertext:
=====
Automated Cryptanalysis System
=====

Enter the ciphertext:
WKLV LV D VHFUHW PHVVDJH

--- ANALYZING CIPHERTEXT ---

Detected Cipher Type: Caesar

--- ATTACKING CIPHER ---

[Caesar Cipher Attack]
Detected Cipher: Caesar
Best Key: 3
Decrypted Text: THIS IS A SECRET MESSAGE
Confidence Score: 10

=====
Analysis Completed
=====
```

Test 4 :

```
Problems @ Javadoc Declaration Console X
<terminated> main [Java Application] C:\Users\HP\.p2\pool\plugins\org.eclipse.j
Enter the ciphertext:
=====
Automated Cryptanalysis System
=====

Enter the ciphertext:
ZIT GSV XZHG RH Z TIVZG VCVZOZO

--- ANALYZING CIPHERTEXT ---

Detected Cipher Type: Unknown

--- ATTACKING CIPHER ---

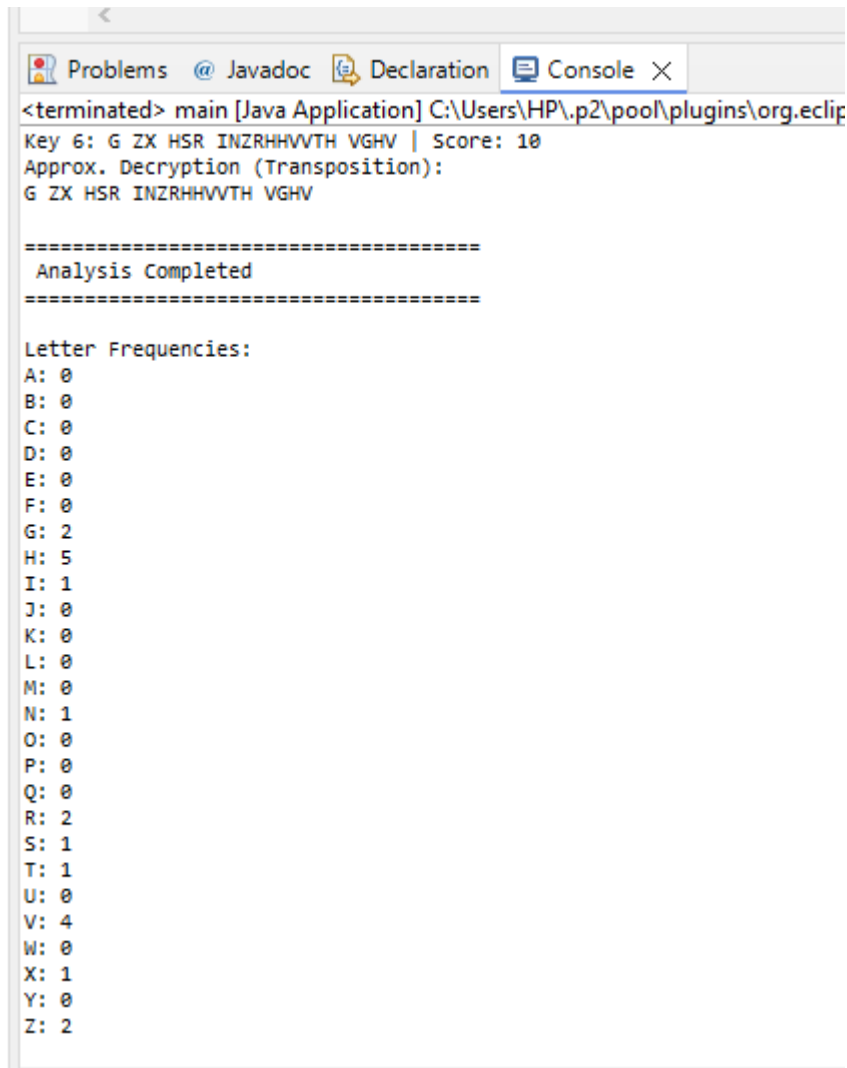
[Substitution Cipher Attempt]
Approx. Decryption (Substitution):
EIS ADT LEOA RO E SITEA THTENEN

[Transposition Cipher Attempt]
Key 2: ZZI TT IGVSZVG XVZCHVGZ ORZHO | Score: 0
Key 3: ZGGI TRV HCG VSZZV O TZXIOZVHZ | Score: 0
Key 4: ZXZVIZ CTHTV GIZG VOSRZZVHGO | Score: 0
Key 5: Z HZOIX GZTZZ O H VGGTCS IVVRVZ | Score: 0
Key 6: ZV TVOI RICTXHVZ Z ZZGHZGOSG Z | Score: 0
Approx. Decryption (Transposition):
ZZI TT IGVSZVG XVZCHVGZ ORZHO

=====
Analysis Completed
=====
```

Frequency Analysis Output

The frequency analyzer counts the occurrence of each alphabet letter in the ciphertext. This helps in identifying patterns used in monoalphabetic substitution ciphers



```
<terminated> main [Java Application] C:\Users\HP\p2\pool\plugins\org.eclip
Key 6: G ZX HSR INZRHHVVTH VGHV | Score: 10
Approx. Decryption (Transposition):
G ZX HSR INZRHHVVTH VGHV

=====
Analysis Completed
=====

Letter Frequencies:
A: 0
B: 0
C: 0
D: 0
E: 0
F: 0
G: 2
H: 5
I: 1
J: 0
K: 0
L: 0
M: 0
N: 1
O: 0
P: 0
Q: 0
R: 2
S: 1
T: 1
U: 0
V: 4
W: 0
X: 1
Y: 0
Z: 2
```

Conclusion

The **Automated Cryptanalysis System for Classical Ciphers** successfully demonstrates how classical encryption techniques such as Caesar, monoalphabetic substitution, and transposition ciphers can be analyzed and broken using computational methods. The system was implemented in Java using the Eclipse IDE and integrates multiple modules including frequency analysis, cipher detection, and automated decryption techniques.

Through testing, the system was able to correctly identify cipher types and recover plaintext from encrypted messages without prior knowledge of the key. This shows how statistical analysis, brute-force search, and pattern recognition can effectively be applied in cryptanalysis.

Overall, the project provides a strong practical understanding of classical cryptography and highlights the importance of modern encryption methods, as classical ciphers can be easily broken using automated techniques.